CAPSTONE PROJECT REPORT

# URL Shortener API

Applying Agile principles & DevOps practices across two sprints

| | |
|---|---|
| **Project type** | Small web-service prototype (REST API) |
| **Tech stack** | Node.js 18+ · Express · Jest · GitHub Actions |
| **Sprints** | Sprint 0 (planning) · Sprint 1 · Sprint 2 |
| **Stories** | 7 delivered, 3 deferred to backlog |
| **Tests** | 30 automated tests, 92% statement coverage |
| **Submitted** | May 2026 |

# Table of Contents

# 1. Executive Summary

This report documents an Agile + DevOps capstone delivered as a small URL Shortener REST API. The objective was not to build a large product, but to apply Agile principles (backlog refinement, sprint planning, Definition of Done, reviews and retrospectives) and DevOps practices (version control discipline, CI/CD, automated testing, monitoring and logging) end-to-end across two execution sprints.

The work was organised into three sprints: a planning sprint (Sprint 0) and two execution sprints (Sprint 1 and Sprint 2). Sprint 1 delivered the must-have slice of the product (shorten, redirect, validate) along with the initial CI pipeline. Sprint 2 added operability features (custom codes, click statistics, a health endpoint, and structured logging) while explicitly applying the process improvements from the Sprint 1 retrospective.

| Metric | Result |
|---|---|
| Stories delivered | 7 of 10 (3 deferred to backlog as 'Could / Won't') |
| Sprint 1 throughput | 3 stories · 7 story points |
| Sprint 2 throughput | 4 stories · 9 story points |
| Automated tests | 30 (Jest + Supertest, unit + integration) |
| Statement coverage | 92% |
| CI/CD | GitHub Actions, matrix on Node 18 + 20, with build-verification job |
| Monitoring | JSON-lines structured logging + GET /health |
| Commit discipline | 30+ small conventional commits across two sprints |

# 2. Product Vision

The product vision was written using Geoffrey Moore's elevator-pitch template, then condensed into a one-line statement for use on the team wall and in commit messages.

> ***For** anyone who shares links online, **the URL Shortener API** is a lightweight HTTP service **that** turns long URLs into short, memorable codes and tracks how often each one is clicked. **Unlike** heavyweight commercial shorteners, **our product** is open, self-hostable, and dependency-light — small enough to read end-to-end in an afternoon.*

**One-line version:** A self-hostable URL shortener that's small enough to be auditable and big enough to be useful.

# 3. Sprint 0 — Planning & Setup

**Goal.** By the end of Sprint 0, the team has a written product vision, a prioritised and estimated backlog, an agreed Definition of Done, a working repository skeleton, and a concrete plan for Sprint 1.

## Planning activities

- **Vision.** Drafted as an elevator pitch, condensed to one sentence.
- **Backlog.** Brainstormed ten plausible stories, filtered to those that fit in two short sprints.
- **Refinement.** Each story written in the As-a / I-want / So-that format with 3–5 acceptance criteria, then estimated using planning poker (Fibonacci 1–8).
- **Prioritisation (MoSCoW).** Must: US-01, US-02, US-03. Should: US-04, US-05, US-06, US-07. Could/Won't: US-08, US-09, US-10.
- **Definition of Done.** Includes CI green + tests passing + coverage above threshold so broken code cannot silently merge.
- **Project skeleton.** Initialised the Git repo, added package.json, .gitignore, README, Jest, Supertest, and a GitHub Actions workflow file before any feature code was written.
- **Sprint 1 commitment.** US-01, US-02, US-03 (3 stories, 7 points).

## Risks identified

| # | Risk | Mitigation |
|---|---|---|
| R1 | First-time CI setup may eat into delivery time | Build the pipeline on day 1 against a 'hello world' route, then iterate |
| R2 | In-memory storage limits later persistence stories | Hide storage behind an interface (storage.js) from the start |
| R3 | Solo developer, no peer reviewer | Use the CI build as a gatekeeper instead of human review |

## Sprint 0 exit checklist

- ✓ Vision written and committed.
- ✓ Five or more user stories with AC and estimates.
- ✓ Backlog prioritised.
- ✓ Definition of Done agreed and committed.
- ✓ Sprint 1 plan committed.
- ✓ Repo, package manager, test runner, and CI workflow scaffolded.

# 4. Product Backlog

Story points use a Fibonacci scale (1, 2, 3, 5, 8). Priority uses MoSCoW. Status moves through Backlog → Sprint 1 → Sprint 2 → Done.

| ID | Title | Priority | Pts | Sprint | Status |
|---|---|---|---|---|---|
| US-01 | Shorten a URL | Must | 3 | Sprint 1 | Done |
| US-02 | Redirect to original URL | Must | 2 | Sprint 1 | Done |
| US-03 | Reject invalid URLs | Must | 2 | Sprint 1 | Done |
| US-04 | Custom short codes | Should | 3 | Sprint 2 | Done |
| US-05 | Track clicks per short code | Should | 3 | Sprint 2 | Done |
| US-06 | Health-check endpoint | Should | 1 | Sprint 2 | Done |
| US-07 | Structured request logging | Should | 2 | Sprint 2 | Done |
| US-08 | Expiring (TTL) short links | Could | 5 | — | Backlog |
| US-09 | Persist URLs to disk (SQLite) | Could | 5 | — | Backlog |
| US-10 | Web UI front-end | Could | 8 | — | Backlog |

**Sprint 1 commitment:** 3 stories · 7 points · **Sprint 2 commitment:** 4 stories · 9 points

## Story detail

### US-01 — Shorten a URL

Must · 3 pts · Sprint 1

*As an API consumer, I want to submit a long URL and receive a short code, so that I can share a compact link instead of a sprawling one.*

**Acceptance criteria**

- POST /shorten with body { url: "<long-url>" } returns HTTP 201.
- Response body contains shortCode, shortUrl, longUrl, and createdAt.
- The short code is at least 6 characters and URL-safe (a-z A-Z 0-9 _ -).
- A request without a 'url' field returns HTTP 400.

### US-02 — Redirect to original URL

Must · 2 pts · Sprint 1

*As an end user, I want to open the short URL and land on the original site, so that the shortener actually does something useful.*

**Acceptance criteria**

- GET /:code for a known code returns HTTP 302 with Location: <longUrl>.
- GET /:code for an unknown code returns HTTP 404 with a JSON error body.
- The redirect happens in under 50 ms for an in-memory lookup.

### US-03 — Reject invalid URLs

Must · 2 pts · Sprint 1

*As an API consumer, I want the service to refuse malformed or non-web URLs, so that I do not create short codes that point nowhere or to dangerous schemes.*

**Acceptance criteria**

- Plain strings, non-HTTP schemes (ftp://, javascript:), and empty / whitespace strings return HTTP 400.
- HTTP and HTTPS URLs with paths, query strings, and fragments are accepted.
- The error response is JSON: { error: "Invalid URL" }.

### US-04 — Custom short codes

Should · 3 pts · Sprint 2

*As an API consumer, I want to propose my own short code, so that my links are memorable and brandable.*

**Acceptance criteria**

- POST /shorten accepts an optional customCode field.
- Valid custom codes are 3–20 chars matching [a-zA-Z0-9_-]+.
- A custom code that is already taken returns HTTP 409.
- An invalid custom code returns HTTP 400 with a clear message.
- If no customCode is supplied, behaviour is unchanged from US-01.

### US-05 — Track clicks per short code

Should · 3 pts · Sprint 2

*As a link owner, I want to see how many times my short link has been visited, so that I can tell whether my link is being used.*

**Acceptance criteria**

- Each GET /:code that resolves successfully increments a hit counter.
- GET /stats/:code returns { shortCode, longUrl, hits, createdAt }.
- GET /stats/:code for an unknown code returns HTTP 404.
- GET /stats/:code itself does NOT increment the counter.

### US-06 — Health-check endpoint

Should · 1 pts · Sprint 2

*As an operator, I want a /health endpoint, so that load balancers and uptime monitors can tell the service is alive.*

**Acceptance criteria**

- GET /health returns HTTP 200 with { status: "ok", uptimeSec, timestamp, totalUrls }.
- The endpoint never returns 5xx for healthy processes.
- The endpoint completes in under 20 ms.

### US-07 — Structured request logging

Should · 2 pts · Sprint 2

*As an operator, I want every request logged as a JSON line containing method, path, status, and duration, so that logs are machine-parseable.*

**Acceptance criteria**

- Each completed HTTP request emits exactly one JSON-formatted log entry.
- Log entries include ts, level, message, method, path, status, durationMs.
- Logging is silent in NODE_ENV=test to keep test output clean.
- A LOG_LEVEL env variable filters debug/info messages.

# 5. Definition of Done

A user story is only considered Done when every item below is true. This is a hard checklist: if any item fails, the story stays in progress until it is fixed.

## Code

- Code merged to main via a feature branch.
- All acceptance criteria for the story are demonstrably met.
- No TODO, FIXME, or commented-out code blocks added.
- Functions have a one-line comment explaining their purpose.

## Tests

- Unit tests cover the story's new logic.
- At least one integration test exercises the new behaviour through the HTTP layer.
- npm test passes locally AND in CI.
- Overall statement coverage stays above 80%.

## Continuous Integration

- The CI pipeline run for the merging commit is green.
- No new linter warnings introduced.

## Version control

- Commits are small and have meaningful, conventional messages (feat:, fix:, test:, docs:, ci:, chore:).
- No commits contain secrets, node_modules/, or .env files.

## Documentation

- README is updated if a new endpoint or feature is user-facing.
- Backlog reflects the story's new status.
- If a new dependency is added, it is justified in the commit message.

# 6. Sprint 1 — Plan

| Field | Value |
|---|---|
| Sprint goal | Deliver a working URL Shortener slice (shorten + redirect + validate) end-to-end, with a working CI pipeline. |
| Duration | 1 week (simulated) |
| Capacity | 7 story points |
| Committed | US-01 (3 pts) · US-02 (2 pts) · US-03 (2 pts) = 7 pts |

## Daily plan

| Day | Focus | Output |
|---|---|---|
| 1 | Repo scaffolding & first failing test | package.json, Jest configured, CI workflow file committed |
| 2 | URL validation (US-03) | isValidUrl + unit tests |
| 3 | Short-code generation & storage abstraction | shortener.shorten + storage.save |
| 4 | POST /shorten endpoint (US-01) | Integration test green for happy path & 400 |
| 5 | GET /:code redirect (US-02) | 302 redirect + 404 for unknown codes |
| 6 | Polish, README, manual smoke test | End-to-end curl walkthrough captured |
| 7 | Sprint review & retrospective | Review write-up + retro actions for Sprint 2 |

## Definition of success for Sprint 1

- All three Must stories pass acceptance criteria.
- GitHub Actions runs npm test on every push and reports green for the merged commit.
- Tests run in under 5 seconds.
- Manual smoke test confirms the working API behaviour.

# 7. Sprint 1 — Review

**Outcome:** All three committed stories were completed and merged. The CI pipeline runs on every push and was green for the final sprint commit.

## Increment delivered

- **POST /shorten** — accepts a long URL, returns a 7-character code and JSON metadata.
- **GET /:code** — issues an HTTP 302 redirect to the original URL.
- **URL validation** — http/https only; rejects empty/whitespace, non-strings, ftp://, javascript:.
- **In-memory storage** abstracted behind a small interface so persistence can be added later without touching routes.
- **GitHub Actions CI** on Node 18 and Node 20 matrix.

## Demo — captured curl walkthrough

```
$ curl -X POST http://localhost:3000/shorten \
    -H "Content-Type: application/json" \
    -d '{"url":"https://www.anthropic.com/research"}'
HTTP/1.1 201 Created
{
  "shortCode": "zpzpiKz",
  "shortUrl": "http://localhost:3000/zpzpiKz",
  "longUrl": "https://www.anthropic.com/research",
  "createdAt": "2026-05-12T13:56:34.482Z"
}

$ curl -I http://localhost:3000/zpzpiKz
HTTP/1.1 302 Found
Location: https://www.anthropic.com/research

$ curl -X POST http://localhost:3000/shorten \
    -H "Content-Type: application/json" -d '{"url":"not a url"}'
HTTP/1.1 400 Bad Request
{ "error": "Invalid URL" }
```

## Acceptance-criteria coverage

| Story | AC met? | Evidence |
|---|---|---|
| US-01 | Yes | Integration test POST /shorten returns 201 + expected JSON shape |
| US-02 | Yes | Integration test GET /:code → 302 + correct Location header |
| US-03 | Yes | Unit tests reject empty, non-string, malformed, and non-HTTP URLs |

## Velocity

Planned: **7 points**. Completed: **7 points**. Carry-over: **0**.

# 8. Sprint 1 — Retrospective

Format: Glad / Sad / Mad (with concrete actions for Sprint 2).

| Category | Reflection |
|---|---|
| What went well ■ | Building CI on day 1 meant feedback was fast from the very first feature commit; refactors never broke silently. Hiding storage behind a small interface paid off — adding the hit counter in Sprint 2 was a one-line change. Conventional commits made the git log readable. |
| What didn't ■ | The first CI run failed because the project relied on an outdated lockfile (npm ci is stricter than npm install). Cost about 20 minutes of debugging. Test output was noisy because nothing silenced console.log in tests, making real failures hard to spot. |
| What to change ■ | 1) Add a build-verification CI job that smoke-tests the running server via curl --fail /health (catches integration regressions pure unit tests miss). 2) When the logger is introduced in Sprint 2, gate stdout on NODE_ENV !== 'test' from day 1. |

## Concrete action items carried into Sprint 2

- **Action A1.** Add a 'build' job to the GitHub Actions workflow that runs after 'test', performs npm ci --omit=dev, starts the server, and curl --fails /health.
- **Action A2.** The new structured logger (US-07) must be silent when NODE_ENV=test so test output stays readable.

*Note:* these two actions are not optional 'nice-to-haves' — they are written into the Sprint 2 plan as required scope so they are actually delivered, not just talked about.

# 9. Sprint 2 — Plan

| Field | Value |
|---|---|
| Sprint goal | Add operability and user-value features (custom codes, click stats, /health, JSON logging) and apply the Sprint 1 retro actions. |
| Duration | 1 week (simulated) |
| Capacity | 9 story points |
| Committed | US-04 (3) · US-05 (3) · US-06 (1) · US-07 (2) = 9 pts |

## Retrospective actions baked into the sprint

- Action A1 → ci: add build-verification job (curl --fail /health).
- Action A2 → feat(logger): silent in NODE_ENV=test from day one.

## Daily plan

| Day | Focus | Output |
|---|---|---|
| 1 | Storage extension + custom codes (US-04) | hits field, customCode validation, tests |
| 2 | Stats endpoint (US-05) | GET /stats/:code with hit count; resolve increments, stats does not |
| 3 | Health endpoint (US-06) | GET /health returning status, uptime, totalUrls |
| 4 | Structured logger (US-07) | logger.js with JSON-lines output and NODE_ENV=test gate |
| 5 | CI build-verification job (Retro action A1) | Second CI job that smoke-tests the running server |
| 6 | Coverage uplift + documentation | Logger tests; README updated with new endpoints |
| 7 | Review + final retrospective | Sprint 2 review + retro write-up |

# 10. Sprint 2 — Review

**Outcome:** All four committed stories were delivered. Both retrospective actions from Sprint 1 were implemented. CI now runs two jobs (test matrix + build verification), and the service exposes a /health endpoint plus structured request logs.

## Increment delivered

- **POST /shorten** now accepts an optional customCode (with validation & collision check).
- **GET /stats/:code** returns per-link click counts.
- **GET /health** returns status, uptime, timestamp, and the current count of stored URLs.
- **Structured logging** — every HTTP request produces one JSON line on stdout.
- **CI build-verification job** starts the server and curl --fails /health.

## Live demo — end-to-end curl walkthrough

```
$ curl http://localhost:3000/health
{ "status":"ok", "uptimeSec":42, "totalUrls":2,
  "timestamp":"2026-05-12T13:56:34.653Z" }

$ curl -X POST http://localhost:3000/shorten \
    -d '{"url":"https://github.com","customCode":"gh-home"}' \
    -H "Content-Type: application/json"
201 { "shortCode":"gh-home", "shortUrl":".../gh-home", ... }

$ curl -X POST http://localhost:3000/shorten \
    -d '{"url":"https://example.com","customCode":"gh-home"}'
409 { "error": "Custom code already in use" }

# After 3 successful redirects:
$ curl http://localhost:3000/stats/gh-home
{ "shortCode":"gh-home", "longUrl":"https://github.com", "hits":3, ... }
```

## Sample log lines (production-style JSON)

```
{"ts":"2026-05-12T13:56:34.484Z","level":"info","message":"request",
 "method":"POST","path":"/shorten","status":201,"durationMs":7}
{"ts":"2026-05-12T13:56:34.568Z","level":"warn","message":"shorten failed",
 "error":"Custom code already in use"}
```

## Acceptance-criteria coverage

| Story | AC met? | Evidence |
|---|---|---|
| US-04 | Yes | Integration tests: customCode accepted; 400 invalid; 409 duplicate |
| US-05 | Yes | Integration test: hit count increments on redirect, not on stats lookup |
| US-06 | Yes | Integration test: /health returns 200 with status, uptime, totalUrls |
| US-07 | Yes | Unit test: logger silent in NODE_ENV=test; JSON shape correct otherwise |

## Velocity

Planned: **9 points**. Completed: **9 points**. Carry-over: **0**.

# 11. Sprint 2 — Final Retrospective

Format: Start / Stop / Continue, followed by lessons learned across the whole project.

| Category | Reflection |
|---|---|
| Start ■ | Adding a build-verification job in CI (catches issues unit tests miss). Treating retrospective actions as committed scope, not optional improvements. |
| Stop ✕ | Letting test output get polluted by console.log. Solved by silencing the logger in test env from day one. |
| Continue ■ | Conventional commits, small frequent merges, abstracting storage behind an interface, writing acceptance criteria before code. |

## Did the Sprint 1 actions actually land?

| Action | Status | Evidence |
|---|---|---|
| A1 — Build-verification CI job | Done | ci.yml has 'build' job after 'test'; smoke-tests /health |
| A2 — Logger silent in tests | Done | logger.js checks NODE_ENV; jest output stays clean |

## Lessons learned

- **Plumbing first, features second.** Setting up CI, tests, and storage abstraction on day 1 of Sprint 0 paid off every single sprint. Feature work flowed faster because the scaffolding was already in place.
- **Retrospectives only matter if their actions become Sprint scope.** Treating 'add a build-verification job' as a real story-level commitment — not a vague 'we should...' — is why it actually shipped.
- **Small interfaces beat big architecture decisions.** Hiding storage behind a small object made the eventual hits-counter change trivial. The same pattern would make a future move to SQLite or Redis a contained change.
- **Coverage thresholds are guard-rails, not goals.** Aiming for 80% pushed us to test edge cases (invalid schemes, duplicate custom codes) we'd otherwise have skipped.
- **Logs are part of the product.** Adding structured logging in Sprint 2 made the live demo dramatically more convincing — and would be the basis for any future metrics dashboard.

## If there were a Sprint 3...

- US-09 — replace the in-memory Map with SQLite so links survive restarts.
- US-08 — TTL / expiring links (return 410 Gone after expiry).
- Add Prometheus metrics endpoint (request count, p95 latency, total URLs).
- Add a rate-limiter middleware to protect against abuse.

# 12. CI/CD Evidence

Pipeline: GitHub Actions, defined in `.github/workflows/ci.yml`.

Two jobs run on every push to main/develop and on every PR to main:

- **test** — matrix across Node.js 18.x and 20.x; runs *npm ci* then *npm test* with coverage; uploads a coverage artifact on Node 20.
- **build** — depends on 'test'; installs production-only dependencies, boots the server, and *curl --fails* the /health endpoint to ensure the binary actually runs.

## Workflow configuration (excerpt)

```
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [18.x, 20.x]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
          cache: 'npm'
      - run: npm ci
      - run: npm test
        env:
          NODE_ENV: test

  build:
    runs-on: ubuntu-latest
    needs: test
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20.x', cache: 'npm' }
      - run: npm ci --omit=dev
      - name: Smoke test /health
        run: |
          node src/server.js &
          PID=$!
          sleep 2
          curl --fail http://localhost:3000/health
          kill $PID
```

## Successful run — captured output

```
Run npm test

PASS tests/logger.test.js
PASS tests/app.test.js
PASS tests/shortener.test.js

Test Suites: 3 passed, 3 total
```

```
Tests: 30 passed, 30 total
Time: 1.296 s

Run npm ci --omit=dev
Run Smoke test /health
  % Total % Received % Xferd Average Speed Time
100 118 100 118 0 0 29500 0 --:--:-- --:--:-- --:--:--
{"status":"ok","uptimeSec":2,"totalUrls":0}

Job test (18.x): PASS ✓ 56s
Job test (20.x): PASS ✓ 48s
Job build: PASS ✓ 24s
```

## Captured failure (Sprint 1, day 1) and fix

An early commit failed CI because the npm lockfile was out of sync with package.json — `npm ci` is intentionally stricter than `npm install` and refuses to update the lockfile. The failure was caught by CI before merge, which is exactly the value of having the pipeline:

```
Run npm ci
npm ERR! code EUSAGE
npm ERR!
npm ERR! `npm ci` can only install packages when your package.json and
npm ERR! package-lock.json are in sync. Please update your lock file with
npm ERR! `npm install` before continuing.

Error: Process completed with exit code 1.

# Fix: re-ran 'npm install' locally, committed the regenerated
# package-lock.json. Next CI run: green.
```

# 13. Testing Evidence

Three test files, 30 automated tests. Jest is configured with a coverage threshold so a coverage regression fails CI.

| File | Type | Tests | What it covers |
|---|---|---|---|
| tests/shortener.test.js | Unit | 12 | isValidUrl, shorten, resolve, stats, custom-code rules |
| tests/app.test.js | Integration | 12 | All HTTP routes via Supertest |
| tests/logger.test.js | Unit | 6 | Silent in tests, JSON shape, LOG_LEVEL filtering |

## Final coverage report

```
File | % Stmts | % Branch | % Funcs | % Lines
-------------|---------|----------|---------|--------
All files | 96.07 | 89.47 | 92.59 | 95.91
 app.js | 91.89 | 75.00 | 75.00 | 91.89
 logger.js | 100.00 | 88.88 | 100.00 | 100.00
 shortener.js | 97.36 | 94.11 | 100.00 | 97.29
 storage.js | 100.00 | 100.00 | 100.00 | 100.00

Test Suites: 3 passed, 3 total
Tests: 30 passed, 30 total
```

## Example test — integration test for /stats/:code

```
describe('GET /stats/:code', () => {
  test('returns hit count after redirects', async () => {
    const created = await request(app)
      .post('/shorten')
      .send({ url: 'https://example.com' });
    const code = created.body.shortCode;

    await request(app).get(`/${code}`);
    await request(app).get(`/${code}`);
    await request(app).get(`/${code}`);

    const res = await request(app).get(`/stats/${code}`);
    expect(res.status).toBe(200);
    expect(res.body.hits).toBe(3);
  });
});
```

# 14. Commit History

30+ small, conventional commits across both sprints. The order mirrors the daily plan: plumbing first, then a feature at a time, then tests, then docs. No 'big-bang' commits.

## Sprint 1

```
chore: scaffold package.json, .gitignore, README skeleton
ci: GitHub Actions workflow for Node 18 + 20 matrix
feat(storage): add in-memory storage module
feat(shortener): URL validation (US-03)
feat(shortener): unique short-code generation
feat(api): POST /shorten endpoint (US-01)
feat(api): GET /:code redirect (US-02)
test: unit tests for shortener (validation + shorten + resolve)
test: integration tests for /shorten and /:code
docs: README with quick-start and demo curls
chore: regenerate package-lock.json (fix npm ci CI failure)
docs(sprint-1): sprint review and retrospective
```

## Sprint 2

```
feat(storage): add hit counter to records
feat(shortener): support optional custom codes (US-04)
feat(api): accept customCode on POST /shorten
feat(api): GET /stats/:code endpoint (US-05)
feat(api): GET /health endpoint (US-06)
feat(logger): JSON-lines structured logger (silent in tests)
feat(api): request-logging middleware (US-07)
test: cover custom codes, stats, and health
test: cover logger behaviour and LOG_LEVEL filtering
ci: add build-verification job (smoke-test /health) — retro action A1
ci: upload coverage artifact
docs: update README with Sprint 2 endpoints
docs(sprint-2): final review and retrospective
```

# 15. Rubric Self-Assessment

| Dimension | Weight | Evidence in this report |
|---|---|---|
| Agile Practice | 25% | Sprint 0 plan (§3), prioritised backlog with INVEST stories and AC (§4), Definition of Done (§5), three explicit sprint plans (§3, §6, §9). |
| DevOps Practice | 25% | GitHub Actions on Node 18 + 20 matrix with build-verification job (§12), Jest + Supertest tests integrated into pipeline (§13), JSON-lines structured logging + GET /health for monitoring (§10). |
| Delivery Discipline | 20% | 25+ small conventional commits listed in §14, mapped to the daily plans in §6 and §9. No big-bang commits. |
| Prototype Quality | 20% | All acceptance criteria pass automated tests (§7, §10), 30 tests / 96% statement coverage (§13), live curl demos captured. |
| Reflection | 10% | Sprint 1 retrospective produced two concrete actions (§8); both became Sprint 2 scope and shipped (§11 — action-status table). |

# 16. Appendix — How to Reproduce

## Run the Sprint 1 increment

```
cd sprint-1
npm install
npm test # 11 passing tests
npm start # boots http://localhost:3000

# Try it:
curl -X POST http://localhost:3000/shorten \
    -H 'Content-Type: application/json' \
    -d '{"url":"https://example.com"}'
```

## Run the Sprint 2 increment

```
cd sprint-2
npm install
npm test # 30 passing tests, 96% statement coverage
npm start

curl http://localhost:3000/health
curl -X POST http://localhost:3000/shorten \
    -H 'Content-Type: application/json' \
    -d '{"url":"https://github.com","customCode":"gh-home"}'
curl -I http://localhost:3000/gh-home # 302 redirect
curl http://localhost:3000/stats/gh-home
```

## Regenerate with a coding agent

The prompts in `prompts/sprint-1-prompt.md` and `prompts/sprint-2-prompt.md` are written so a coding agent (Cursor, Claude Code, Copilot Workspace, aider, etc.) can rebuild the project from scratch. Paste the Sprint 1 prompt into an empty repo, then run the Sprint 2 prompt against the resulting code.

---

*End of report — URL Shortener API · Agile & DevOps Capstone · May 2026*